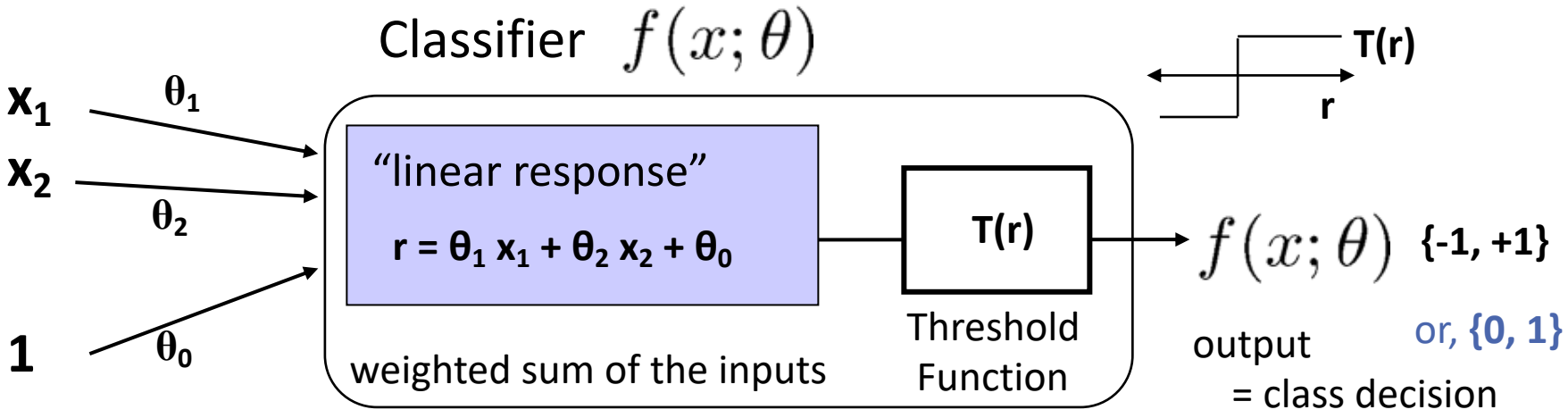
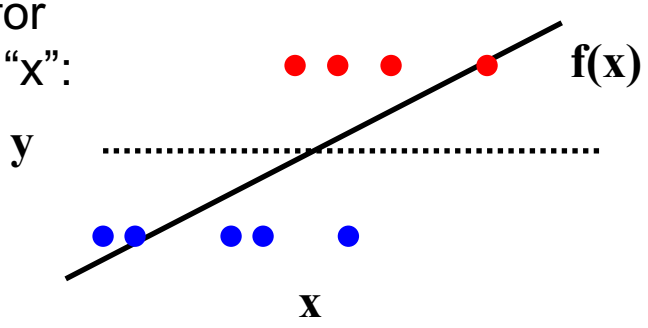
Perceptron Classifier

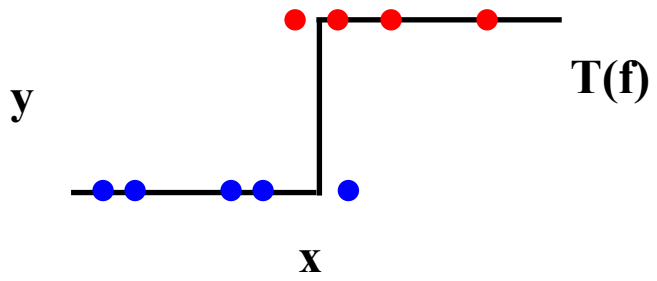



```
r = X.dot( theta.T );       # compute linear response
Yhat = (r > 0)          # predict class 1 vs 0
Yhat = 2*(r > 0)-1      # or "sign": predict +1 / -1
# Note: typically convert classes to "canonical" values 0,1,…
# then convert back ("learner.classes[c]") after prediction
```

Perceptron Classifier

Visualizing for one feature “x”:





Perceptron Decision Boundary

The perceptron is defined by the decision algorithm:

$$f(x;\theta) = \begin{cases} +1 & \text{if } \theta \cdot x^T > 0 \\ -1 & \text{otherwise} \end{cases}$$

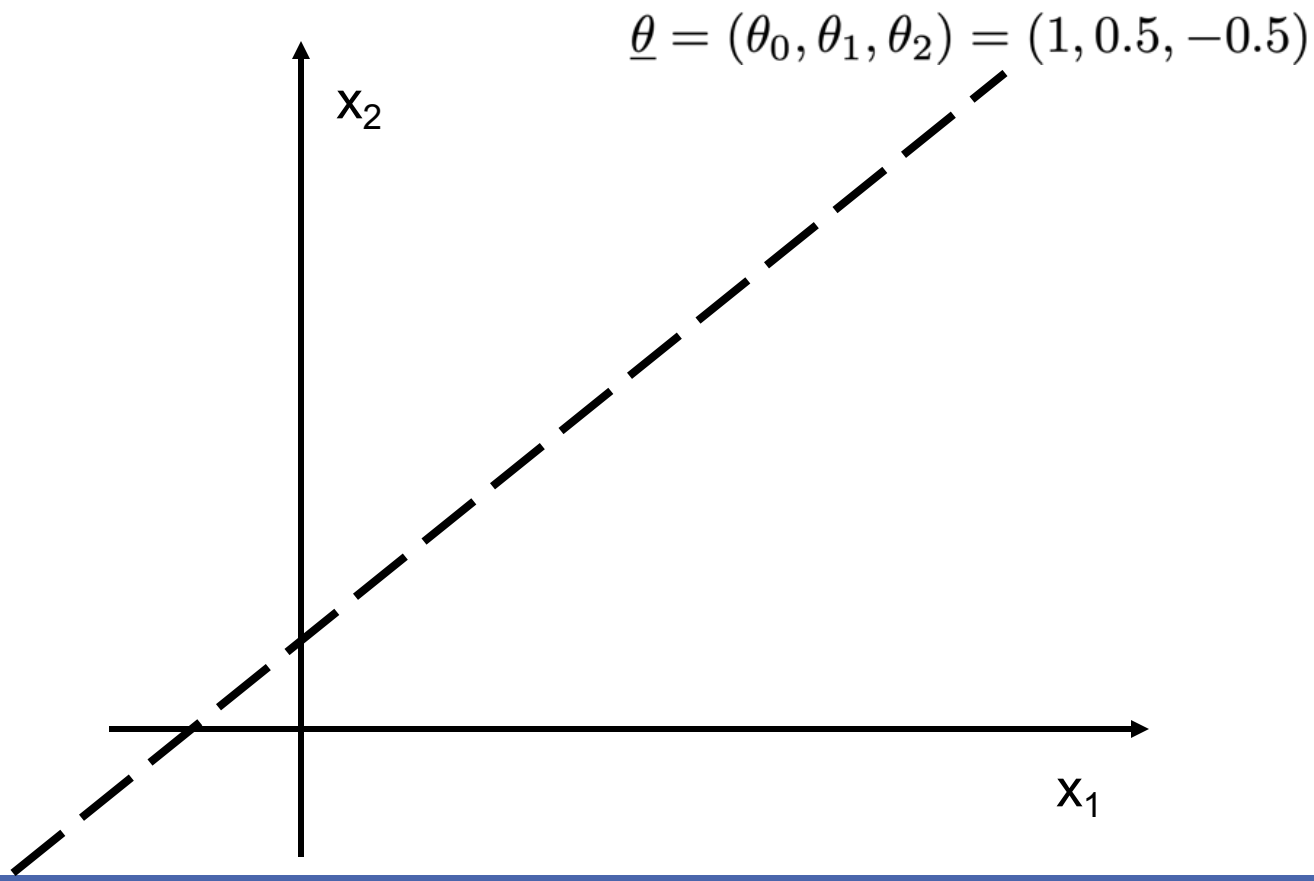
Perceptron represents a hyperplane decision surface in d-dimensional space

- A line in 2D, a plane in 3D, etc.

The equation of the hyperplane is given by $\theta \cdot x^T = 0$

This defines the set of points that are exactly on the boundary.

Example: Linear Decision Boundary

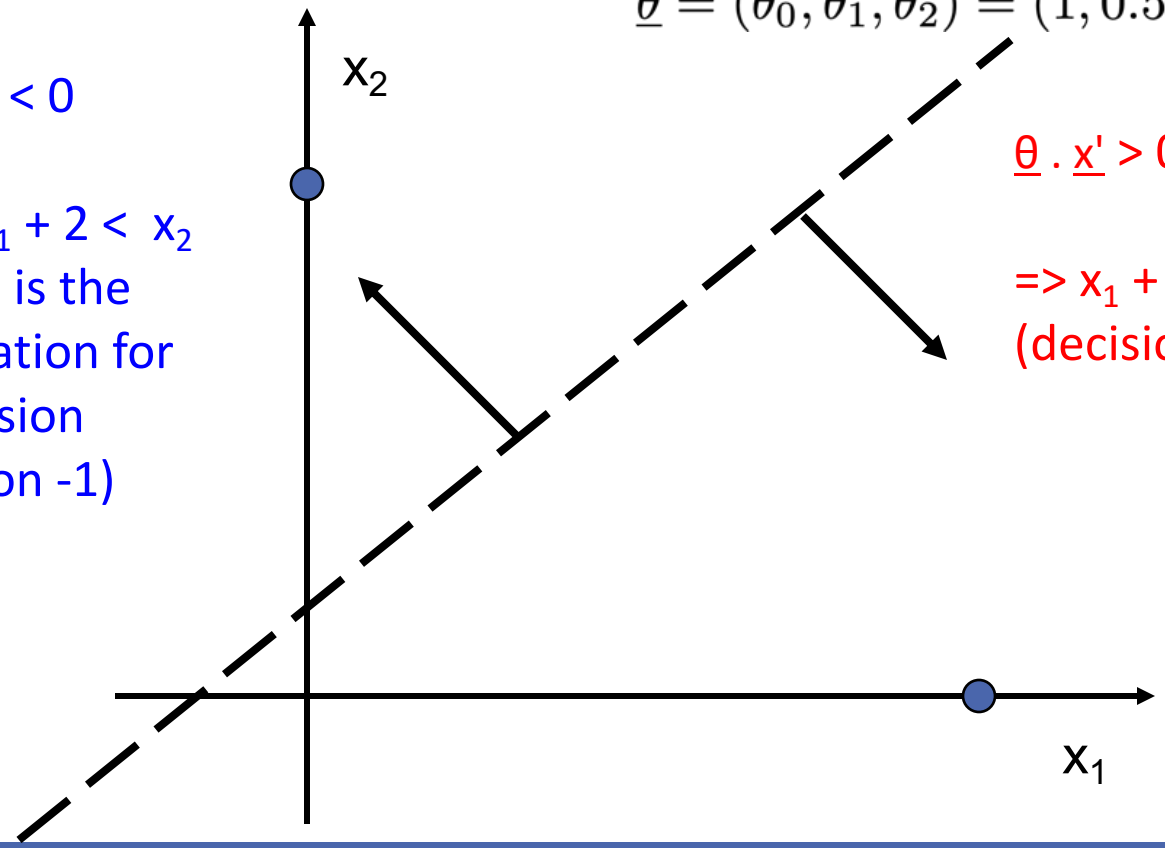


Example: Linear Decision Boundary

$\underline{\theta} = (\theta_0, \theta_1, \theta_2) = (1, 0.5, -0.5)$

$\underline{\theta} \cdot \underline{x}' < 0$

$\Rightarrow x_1 + 2 < x_2$
(this is the equation for decision region -1)

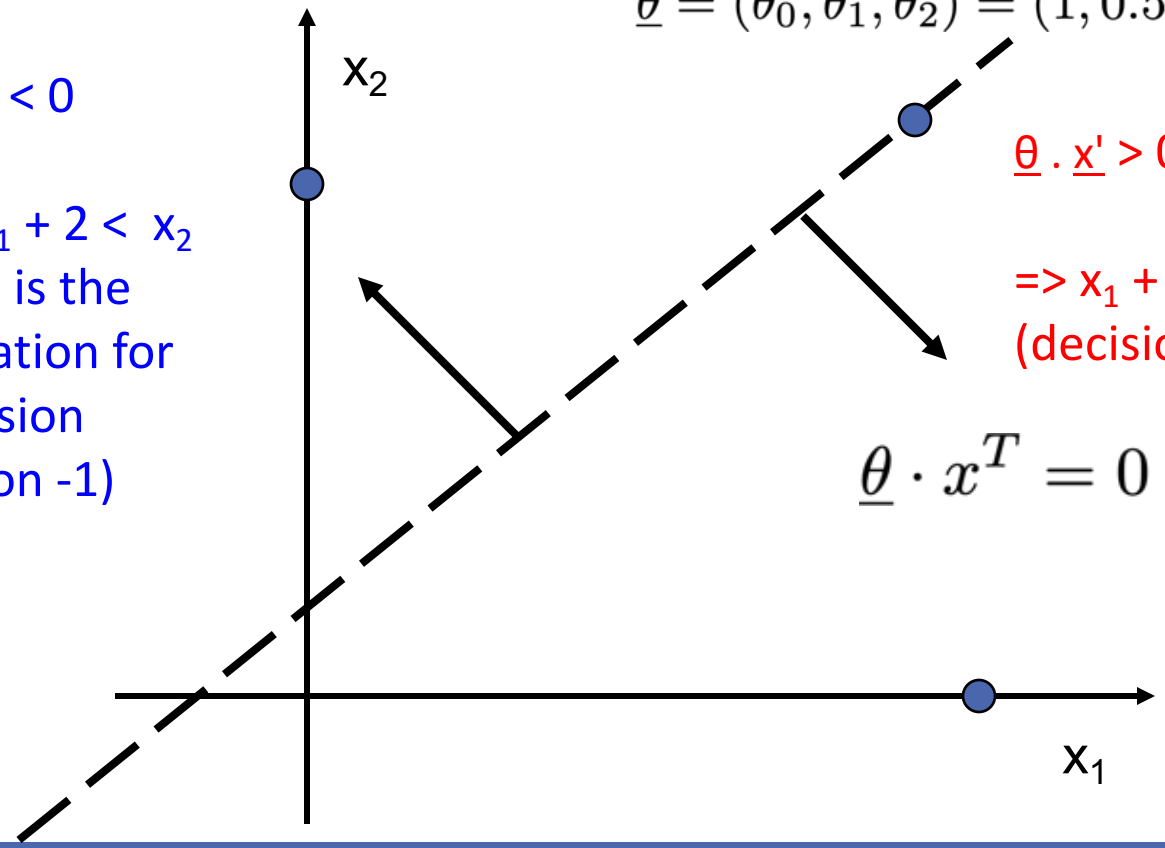


$\underline{\theta} \cdot \underline{x}' > 0$

$\Rightarrow x_1 + 2 > x_2$
(decision region +1)

Example: Linear Decision Boundary

$\underline{\theta} = (\theta_0, \theta_1, \theta_2) = (1, 0.5, -0.5)$



$\underline{\theta}$. $\underline{x'}$ < 0

=> $x_1 + 2 < x_2$
(this is the equation for decision region -1)

$\underline{\theta}$. $\underline{x'}$ > 0

=> $x_1 + 2 > x_2$
(decision region +1)

$$\underline{\theta} \cdot x^T = 0 \Leftrightarrow x_2 = x_1 + 2$$

Separability

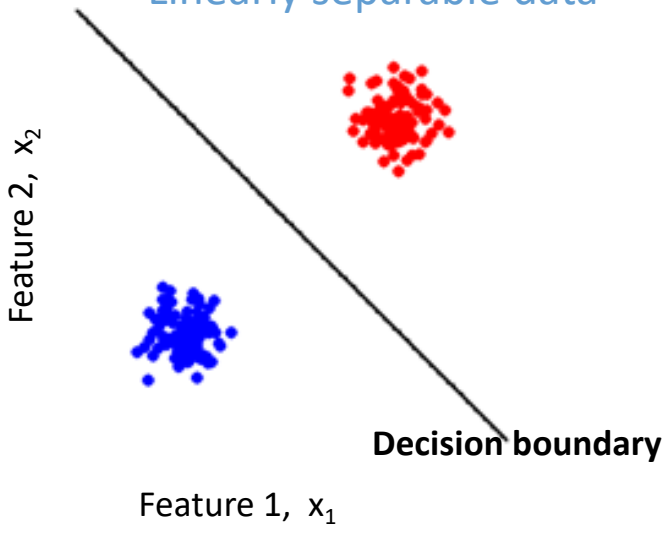
A data set is separable by a learner if

- There is some instance of that learner that correctly predicts all data points

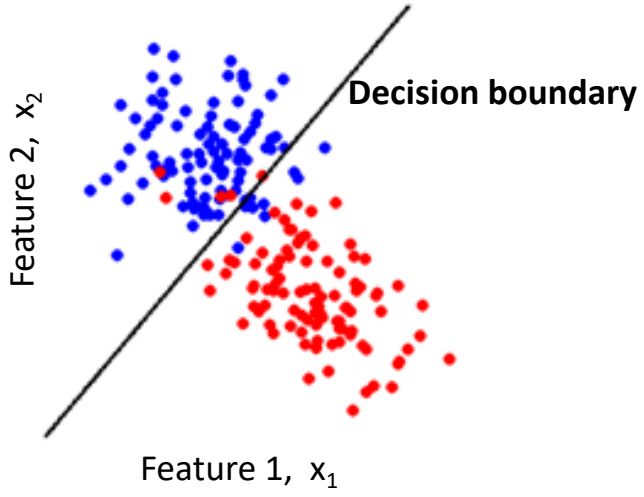
Linearly separable data

- Can separate the two classes using a straight line in feature space
- in 2 dimensions the decision boundary is a straight line

Linearly separable data



Linearly non-separable data



Origins of class overlap

Means that the data are not perfectly predictable by the model

Same observation values possible under both classes

- High vs low risk; features {age, income}
- Benign/malignant cells look similar
- …

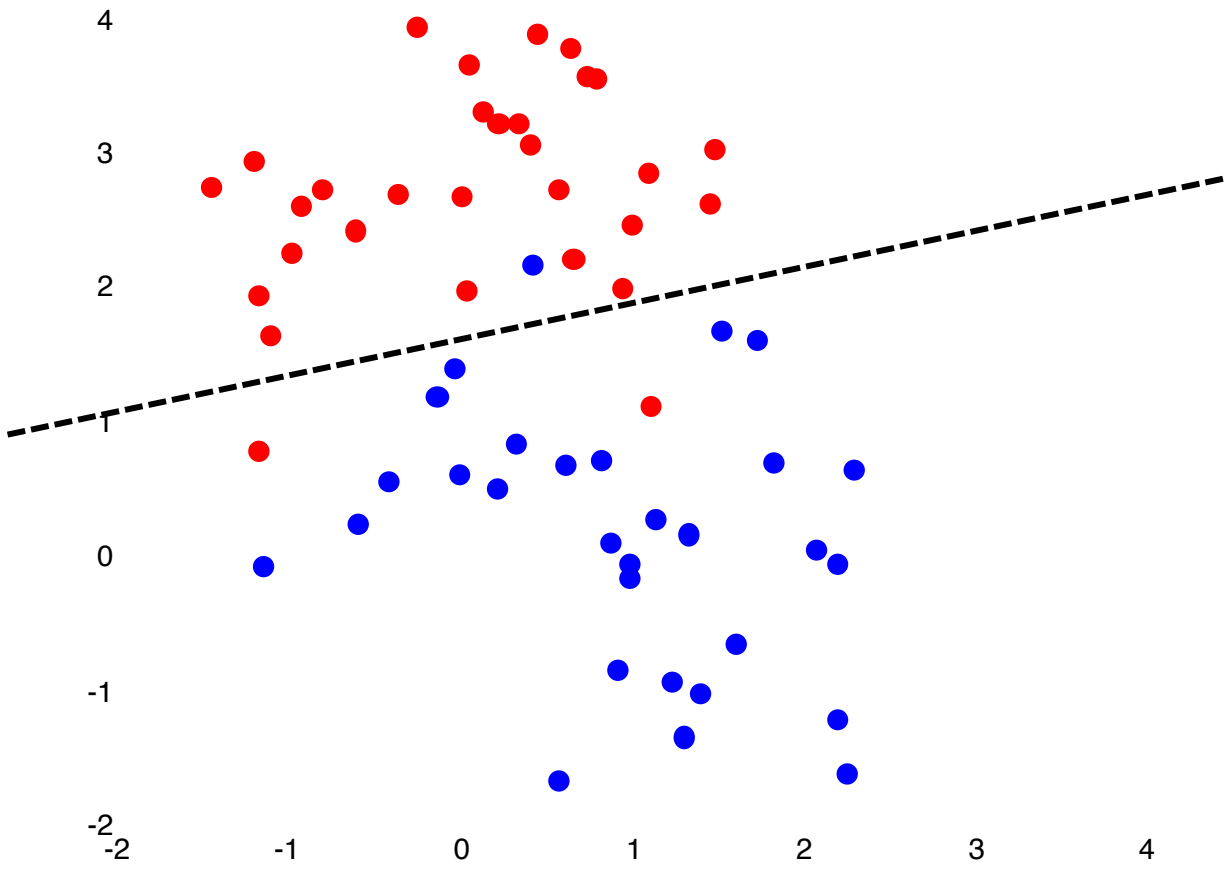
Common in practice

May not be able to perfectly distinguish between classes

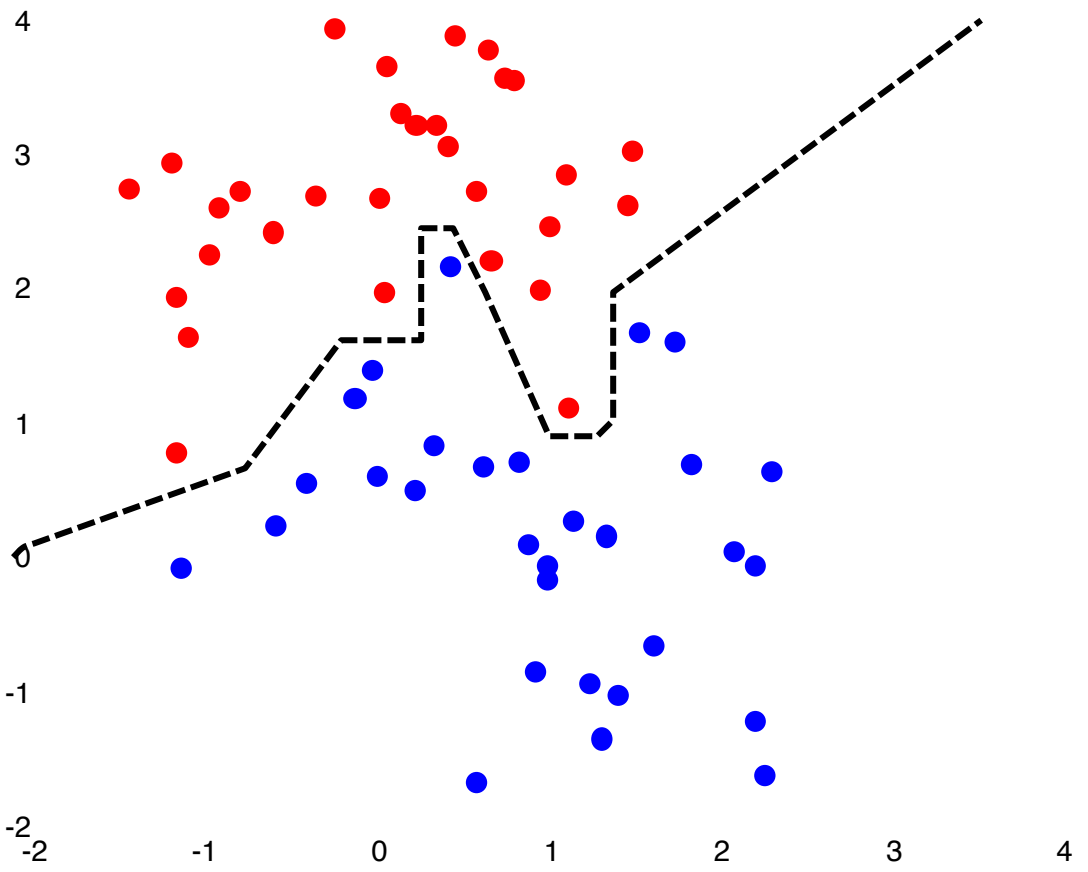
- Maybe with more features?
- Maybe with more complex classifier?

Otherwise, may have to accept some errors

Example: not *linearly* separable



…but separable with *non-linear* decision boundary



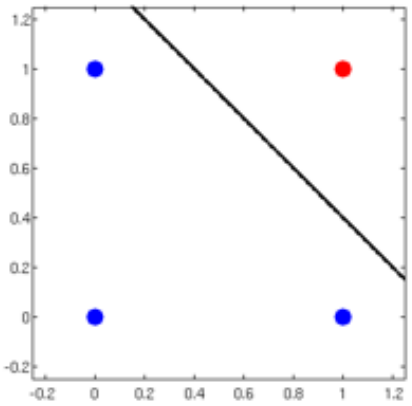
Representational Power of Perceptrons

What mappings can a perceptron represent perfectly?

- A perceptron is a linear classifier
- thus it can represent any mapping that is linearly separable
- some Boolean functions like AND (on left)
- but not Boolean functions like XOR (on right)

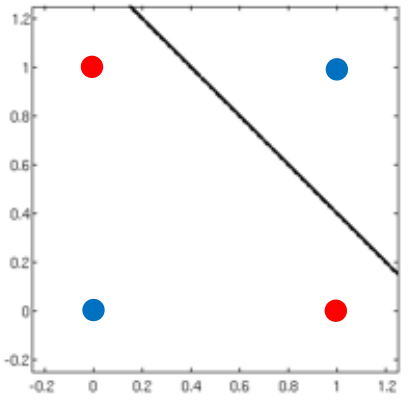
"AND"

x_1	x_2	y
0	0	-1
0	1	-1
1	0	-1
1	1	1



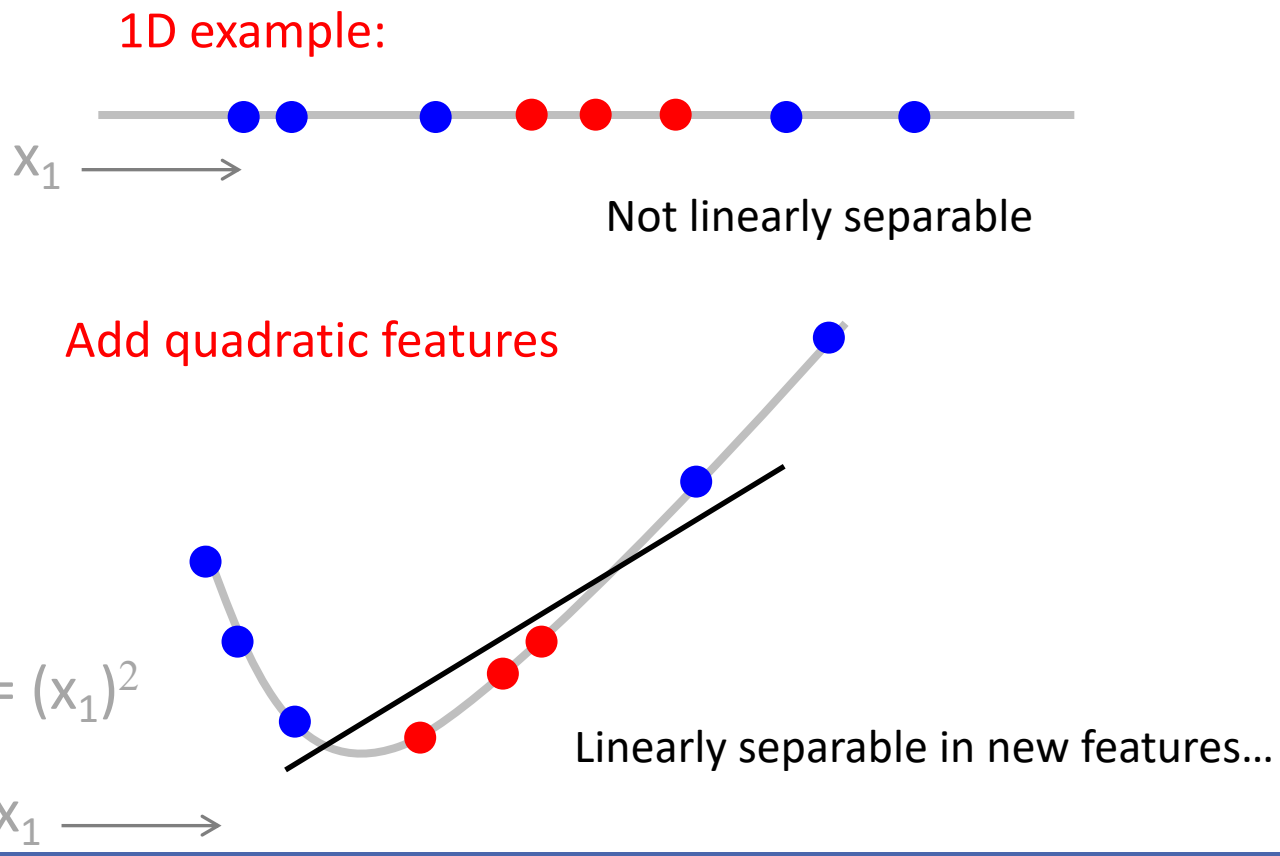
"XOR"

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	-1



Adding features

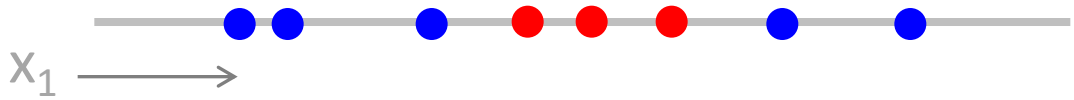
Linear classifier can't learn some functions



Adding features

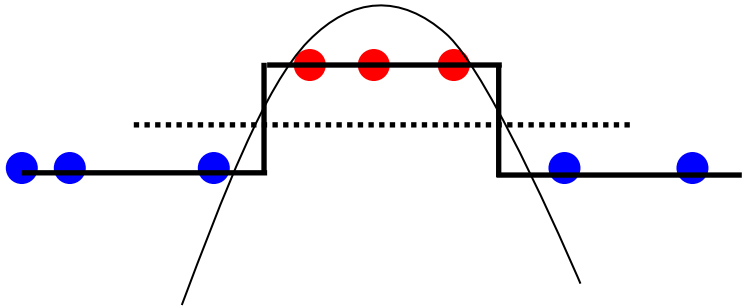
Linear classifier can't learn some functions

1D example:



Not linearly separable

Quadratic features, visualized in original feature space:



$y = T(a x^2 + b x + c)$

More complex decision boundary: $ax^2+bx+c = 0$

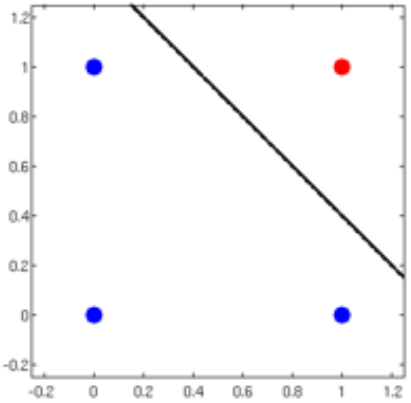
Representational Power of Perceptrons

What mappings can a perceptron represent perfectly?

- A perceptron is a linear classifier
- thus it can represent any mapping that is linearly separable
- some Boolean functions like AND (on left)
- but not Boolean functions like XOR (on right)

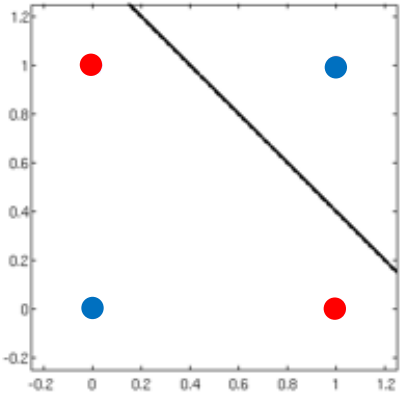
"AND"

x_1	x_2	y
0	0	-1
0	1	-1
1	0	-1
1	1	1



"XOR"

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	-1



What kinds of polynomial features would we need to learn the data on the right?

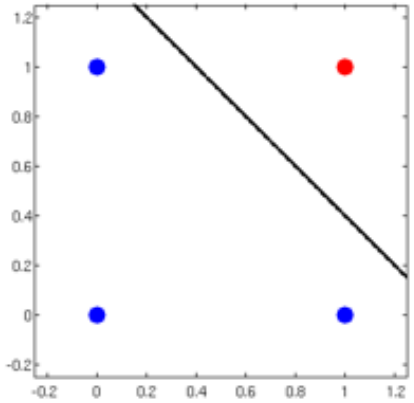
Representational Power of Perceptrons

What mappings can a perceptron represent perfectly?

- A perceptron is a linear classifier
- thus it can represent any mapping that is linearly separable
- some Boolean functions like AND (on left)
- but not Boolean functions like XOR (on right)

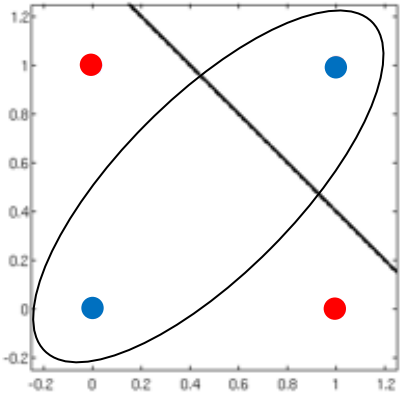
"AND"

x_1	x_2	y
0	0	-1
0	1	-1
1	0	-1
1	1	1



"XOR"

x_1	x_2	y
0	0	-1
0	1	1
1	0	1
1	1	-1



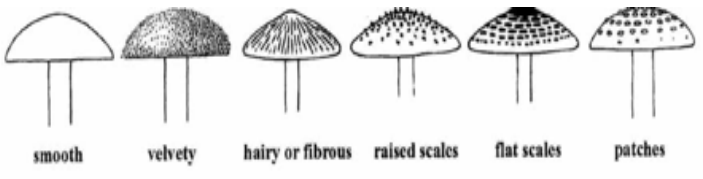
What kinds of polynomial features would we need to learn the data on the right?

Ellipsiodal decision boundary: $a\,x_1^2 + b\,x_1 + c\,x_2^2 + d\,x_2 + e\,x_1x_2 + f = 0$

Feature representations

Features are used in a linear way

- Learner is dependent on representation



Ex: poisonous mushroom classification

- Discrete features
- Mushroom surface: {fibrous, grooves, scaly, smooth}
- Probably not useful to use x = {1, 2, 3, 4} (this would suggest an ordering)
- Better: 1-of-K, or 'one-hot' representation x = { [1000], [0100], [0010], [0001] }
- Introduces more parameters, but a more flexible relationship (no ordering)

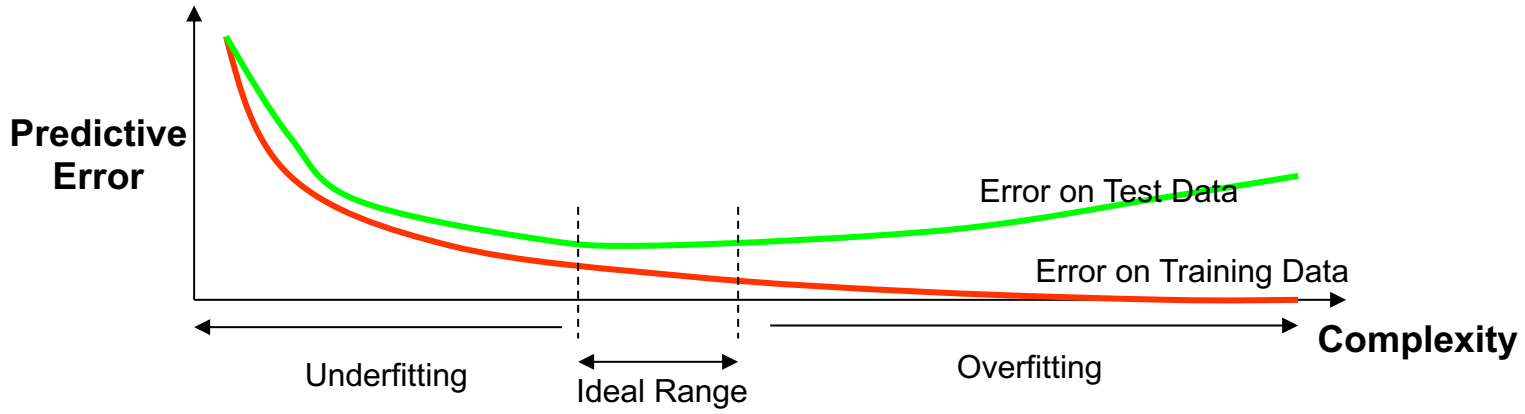
Effect of dimensionality

The Good

- Separation is easier in higher dimensions (for fixed # of data m)
- Increase the number of features, and even a linear classifier will eventually be able to separate all the training examples!

The Bad

- More features can increase our chance of overfitting
- Increasingly complex decision boundaries can eventually get all the training data right, but it doesn't necessarily generalize well for test data…



Summary

Perceptron as a linear classifier

- Converts linear response into a discrete (e.g., binary) class value
- Has a linear decision boundary

Separability

- = the ability of a learner to correctly classify all training data
- Limits of the representational power of a perceptron

Adding features

- Complex features => Complex surfaces
- Help to separate the two classes
- Potential for overfitting

Outline

Perceptrons

Perceptron Learning

Other Linear Classifiers

Multi-Class Classifiers

Learning the Classifier Parameters

Learning from Training Data:

- training data = labeled feature vectors
- Find parameter values that predict well (low error)
  - error is estimated on the training data
  - "true" error will be on future test data

Define an error / loss function $J(\theta)$:

- Classifier error rate (for a given set of weights θ and labeled data)

Minimize this loss function (or, maximize accuracy)

- An optimization or search problem over the vector $(\theta_1, \theta_2, \theta_0)$

Training a linear classifier

How should we measure error?

- Natural measure = "fraction we get wrong" (error rate)

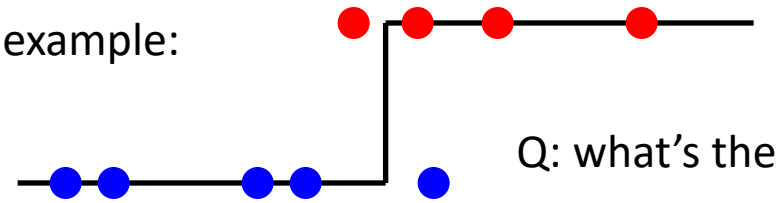
$$\text{err}(\theta) = \frac{1}{m}\sum_{i=1}^{m} \mathbb{I}[y^{(i)} \neq f(x^{(i)};\theta)] \qquad \mathbb{I}[\hat{y} \neq y] = \begin{cases} 1 & \hat{y} \neq y \\ 0 & \text{else} \end{cases}$$

```
Yhat = np.sign( X.dot( theta.T ) );  # predict class  (+1/-1)
err = np.mean( Y != Yhat )          # count errors: empirical error rate
```

But, hard to train via gradient descent

- Not continuous
- As decision boundary moves, errors change abruptly

1D example:

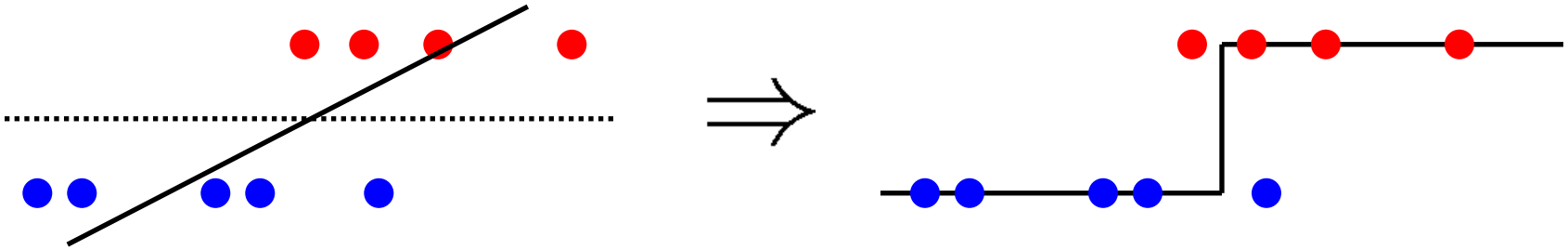


T(f) = -1 if f < 0
T(f) = +1 if f > 0

Q: what's the error?

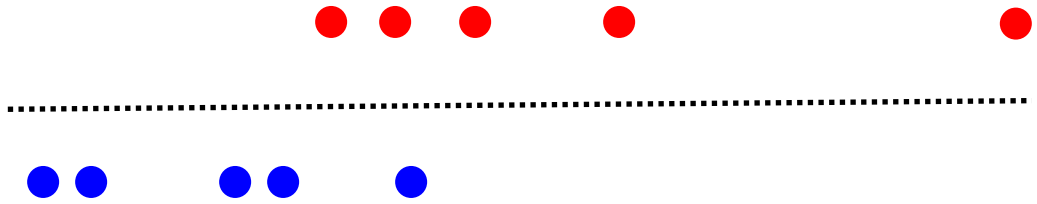
Linear regression?

Simple option: set θ using linear regression



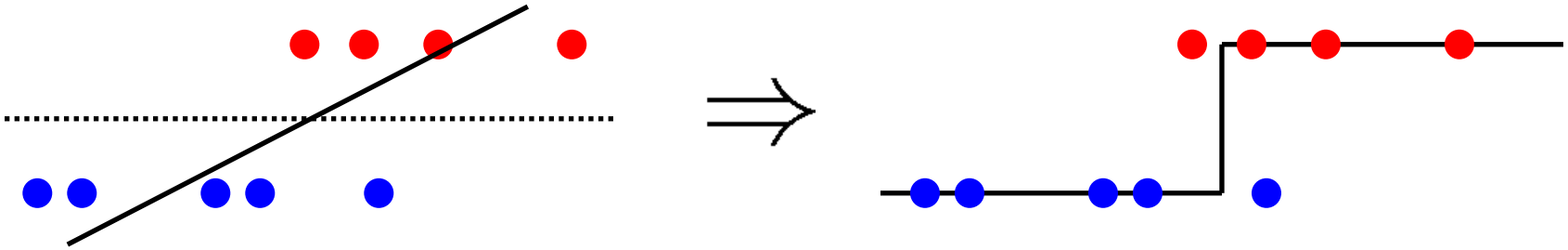
Question: what is potentially wrong with this? Take some time to think.

- Hint: how will the solution be affected by an “easy” point



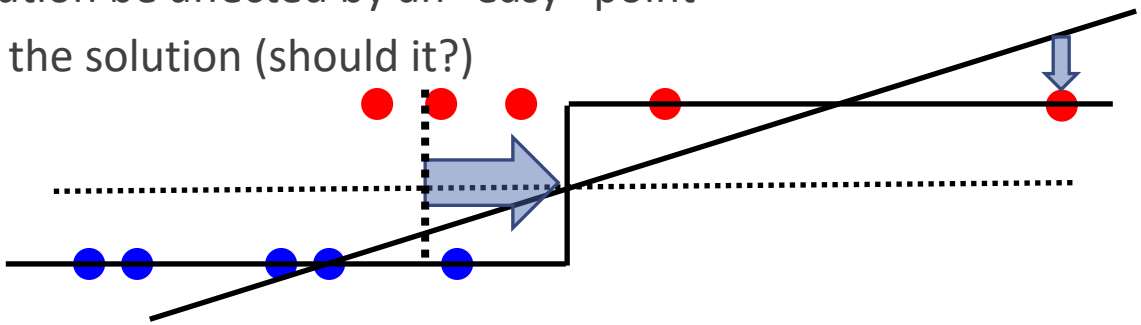
Linear regression?

Simple option: set θ using linear regression



Question: what is potentially wrong with this? Take some time to think.

- Hint: how will the solution be affected by an “easy” point
- Answer: MSE distorts the solution (should it?)



Perceptron algorithm

Perceptron algorithm: an SGD-like algorithm

while ¬ done:

for each data point j:

$$\hat{y}^{(j)} = \text{sign}(\theta \cdot x^{(j)})$$ (predict output for point j)

$$\theta \leftarrow \theta + \alpha(y^{(j)} - \hat{y}^{(j)})x^{(j)}$$ ("gradient-like" step)

Very similar to linear regression + MSE cost

- Identical update to SGD for MSE except the error uses thresholded $\hat{y}^{(j)}$ instead of linear response θ x' so:
  1. For correct predictions, $y(j) - \hat{y}(j) = 0$
  2. For incorrect predictions, $y(j) - \hat{y}(j) = \pm 2$

"adaptive" linear regression: correct predictions stop contributing

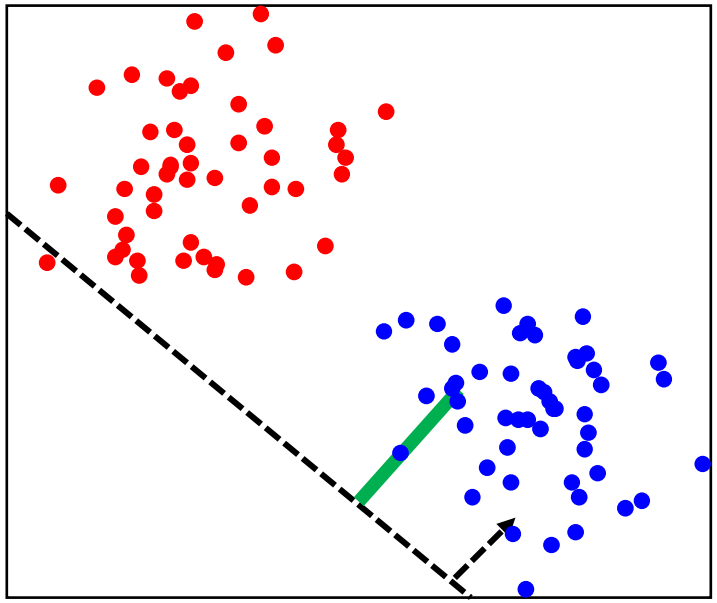
Perceptron algorithm

while ¬ done:

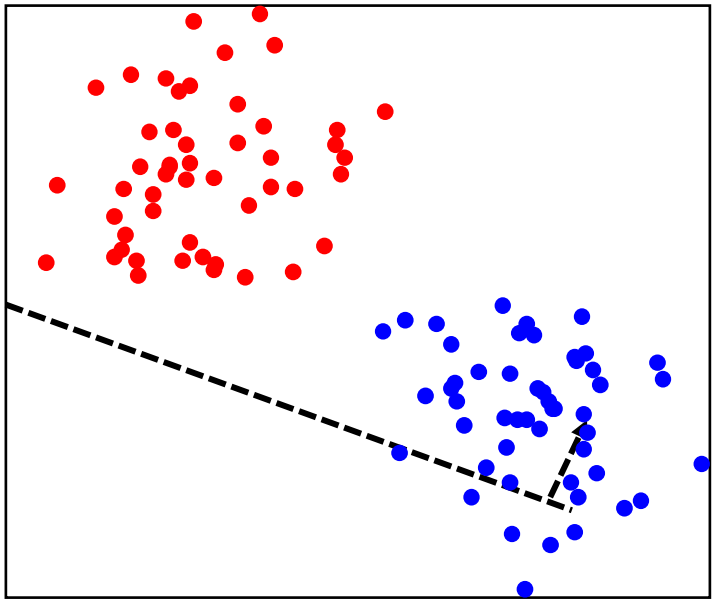
for each data point j:

$$\hat{y}^{(j)} = \text{sign}(\theta \cdot x^{(j)})$$ (predict output for point j)

$$\theta \leftarrow \theta + \alpha(y^{(j)} - \hat{y}^{(j)})x^{(j)}$$ ("gradient-like" step)



$y^{(j)}$ predicted **incorrectly**: update weights



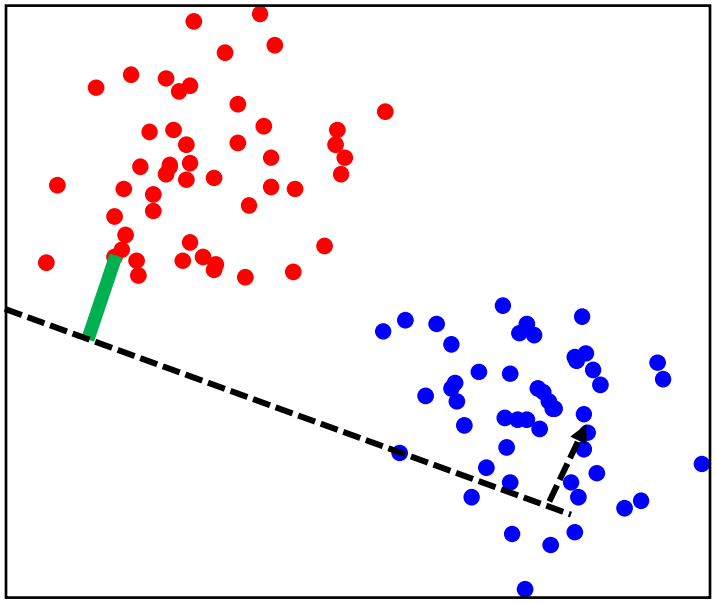
Perceptron algorithm

while ¬ done:

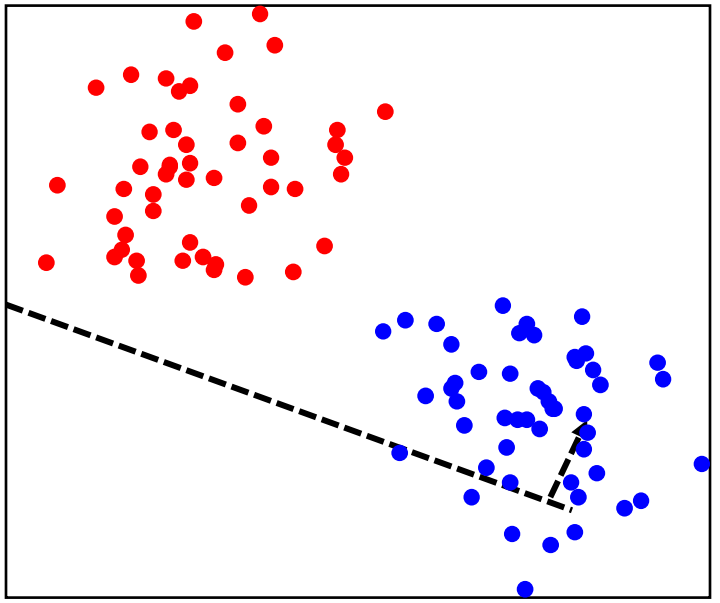
for each data point j:

$$\hat{y}^{(j)} = \text{sign}(\theta \cdot x^{(j)})$$ (predict output for point j)

$$\theta \leftarrow \theta + \alpha(y^{(j)} - \hat{y}^{(j)})x^{(j)}$$ ("gradient-like" step)



$y^{(j)}$ predicted **correctly**: no update



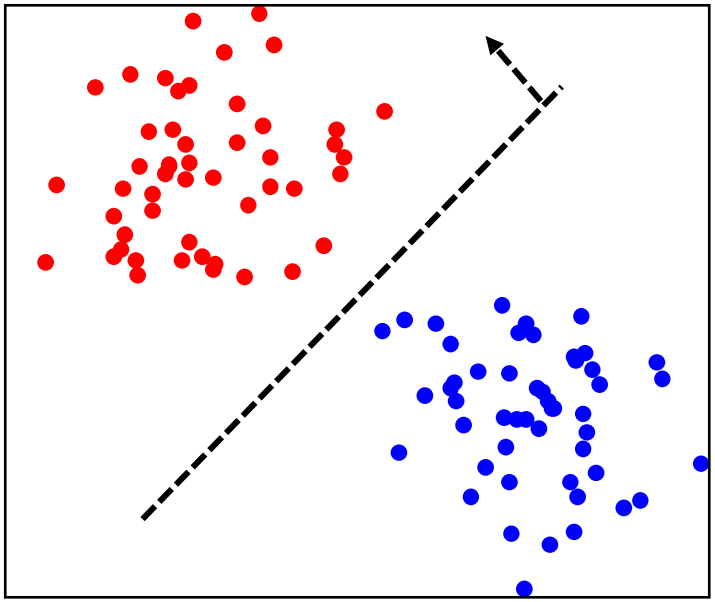
Perceptron algorithm

while ¬ done:

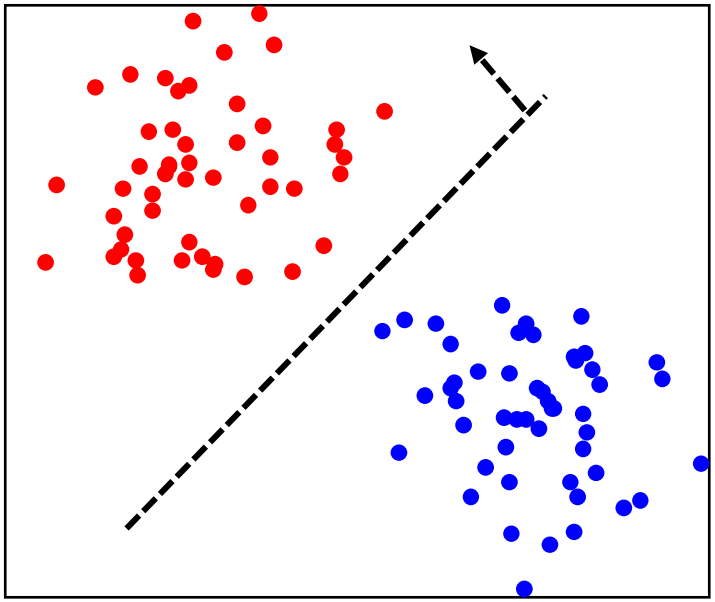
for each data point j:

$$\hat{y}^{(j)} = \text{sign}(\theta \cdot x^{(j)})$$ (predict output for point j)

$$\theta \leftarrow \theta + \alpha(y^{(j)} - \hat{y}^{(j)})x^{(j)}$$ ("gradient-like" step)



$y^{(j)}$ predicted **correctly**: no update



Outline

Perceptrons

Perceptron Learning

Other Linear Classifiers

Multi-Class Classifiers

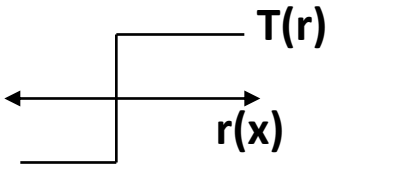
Surrogate loss functions

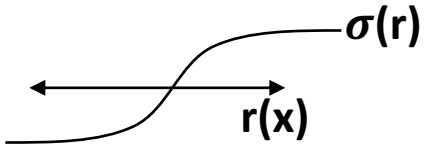
Another solution: use a "smooth" loss

- e.g., approximate the threshold function
- Usually some smooth function of distance
  - Example: logistic "sigmoid", looks like an "S"
- Now, measure e.g. MSE

$$J(\underline{\theta}) = \frac{1}{m}\sum_{j}\left(\sigma(\, r(x^{(j)})\,) - y^{(j)} \right)^2$$

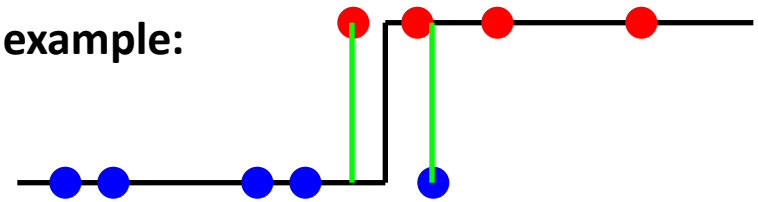
- Far from the decision boundary: |r(.)| large, small error
- Nearby the boundary: |r(.)| near 1/2, larger error

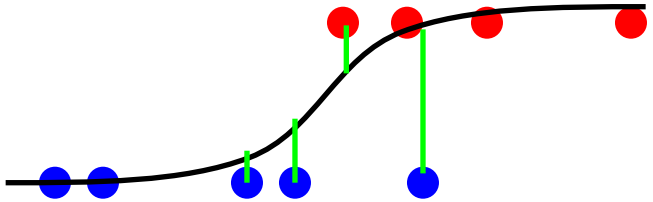




Class y = {0, 1} …

1D example:





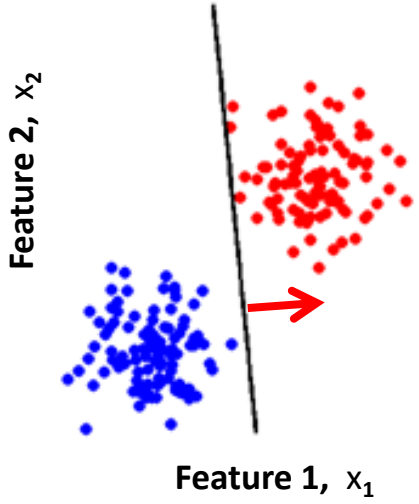
Classification error = 2/9

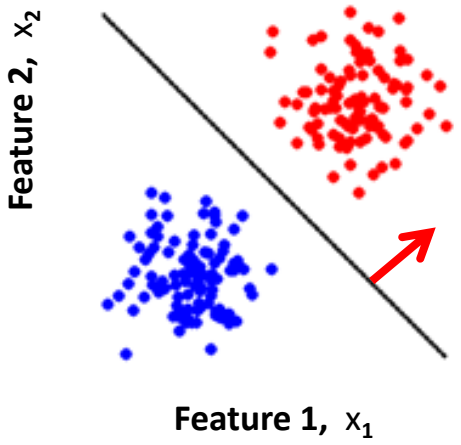
MSE = $(0^2 + .1^2 + .2^2 + .25^2 + .05^2 + \ldots)/9$

Widening the class margin

Which decision boundary is "better"?

- Both have zero training error (perfect training accuracy)
- But, one of them seems intuitively better…





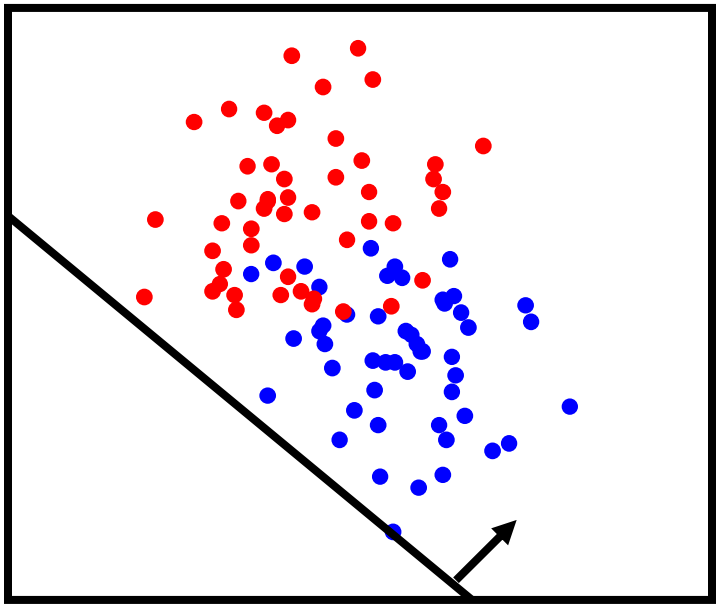
Side benefit of many "smoothed" error functions

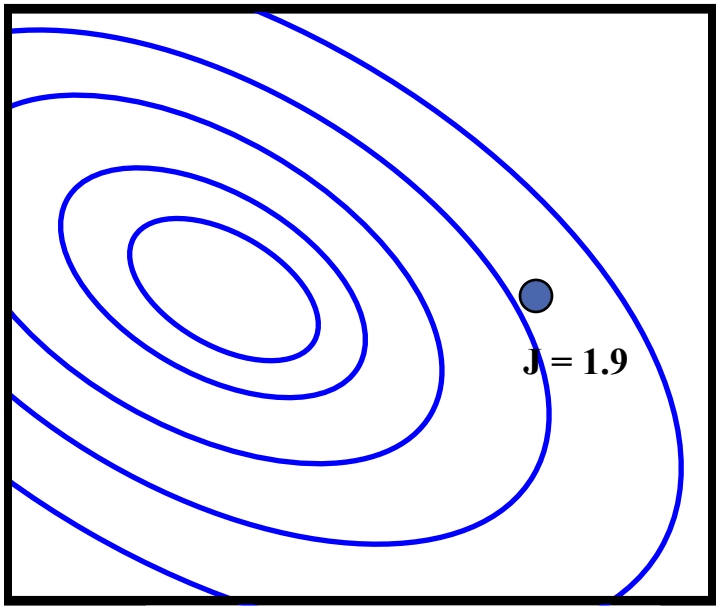
- Encourages data to be far from decision boundary (in contrast to, e.g., Perceptron alg.)
- See more examples of this principle later...

Training the Classifier

Once we have a smooth measure of quality, we can find the "best" settings for the parameters of

$$J(\underline{\theta}) = \frac{1}{m}\sum_{j}\left(\sigma(r(x^{(j)})) - y^{(j)} \right)^2$$

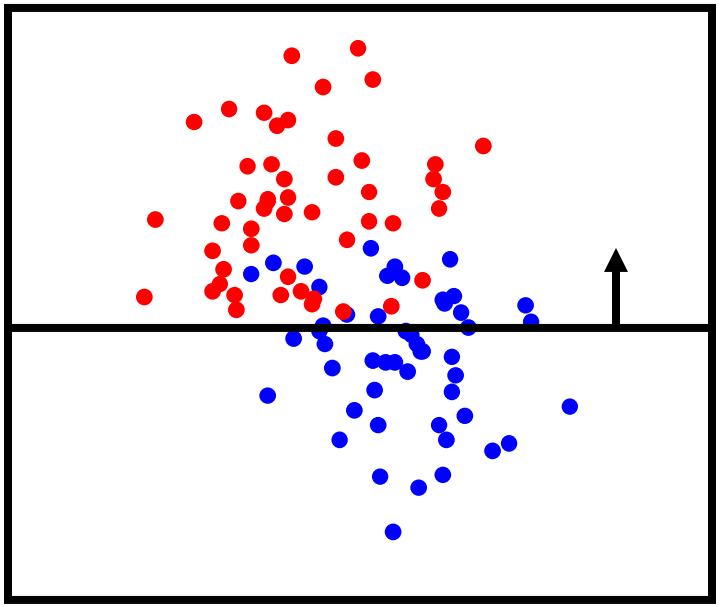


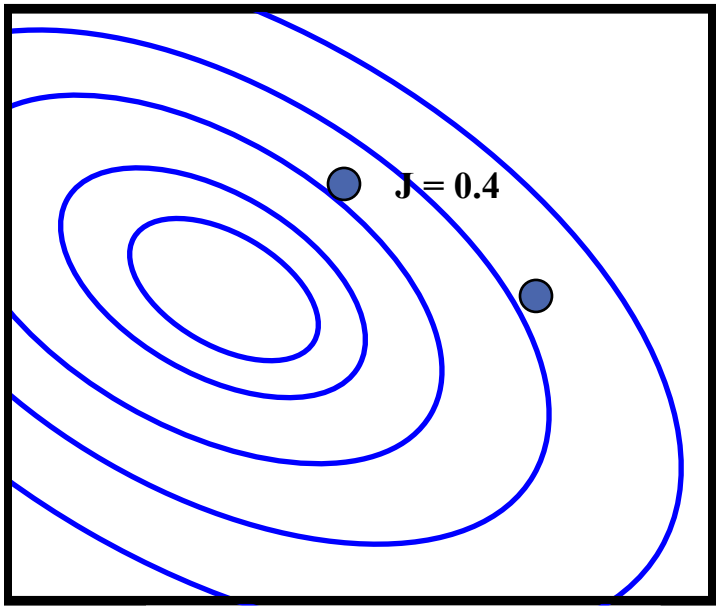


Training the Classifier

Once we have a smooth measure of quality, we can find the "best" settings for the parameters of

$$J(\underline{\theta}) = \frac{1}{m}\sum_{j}\left(\sigma(\, r(x^{(j)})\,) - y^{(j)} \right)^2$$

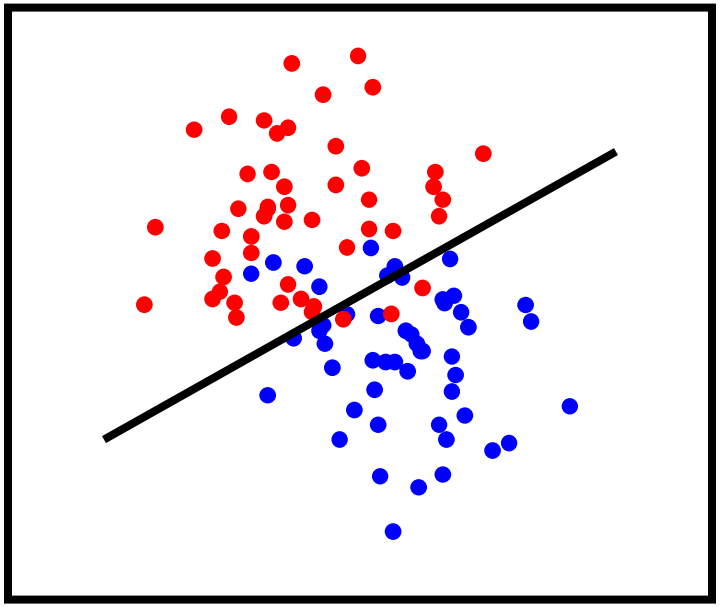


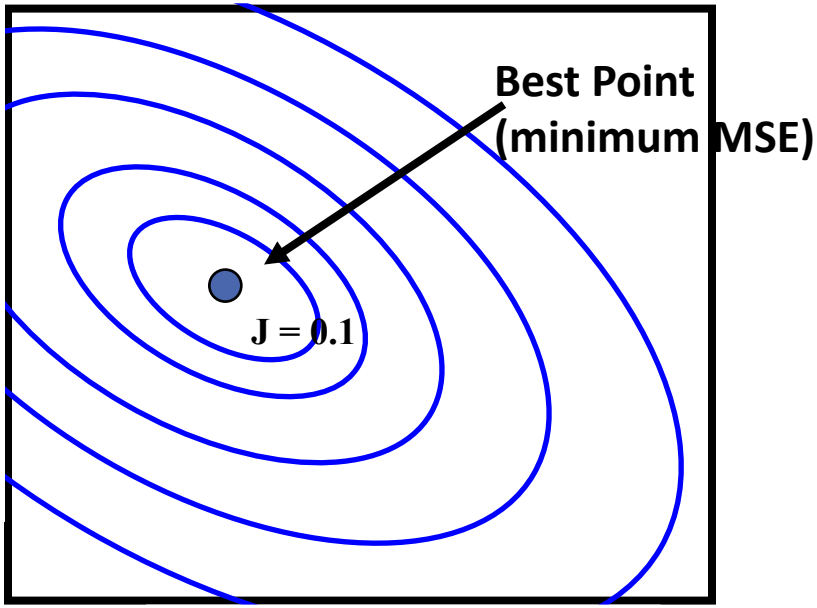


Training the Classifier

Once we have a smooth measure of quality, we can find the "best" settings for the parameters of

$$J(\underline{\theta}) = \frac{1}{m}\sum_{j}\left(\sigma(\, r(x^{(j)})\,) - y^{(j)} \right)^2$$





Finding the Best MSE

As in linear regression, this is now just optimization

Many Possible Methods:

Gradient descent

- Improve loss by small changes in parameters ("small" = learning rate)

Or, use your favorite optimization algorithm…

- Coordinate descent
- Stochastic gradient descent
- Adam/AdaGrad/RMSprop

Gradient Descent